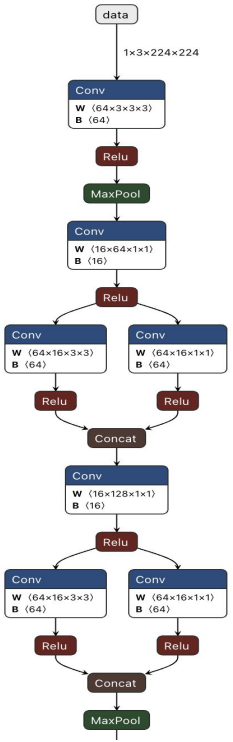
Inference Prediction of ONNX Models

Siddharth Narsipur, Henry Liu

Background: ONNX

- ONNX is an open-source format for representing ML models as a graph.

- Each node in the graph defines an operation. Edges define data flow between nodes.



ONNX

Background: ONNX Runtime

- Most popular ONNX inference engine (20k+ stars on Github) with strong multi-provider (CUDA, ROCm, CPU) support.

- Models can be optimized across four variants.
  - ORT.DISABLE_ALL
  - ORT.ENABLE_BASIC
  - ORT.ENABLE_EXTENDED
  - ORT.ENABLE_ALL

Motivation

- Goal 1: Which optimization level is right for your machine learning model?

- Goal 2: Predict ONNX model performance on a target machine you currently do not have access to.

- Goal 3: Can we predict a model's performance using only static features? Which static features are important in predicting a model's performance?

Project Stages

1: Build dataset

↓

2: Collect inference data

↓

3: Train models

↓

4: Evaluation

1: Build Dataset

Data Source

- ONNX Model Zoo on Hugging Face is the largest collection of open-source ONNX models, curated by the maintainers.

- 2,326 ONNX Models including computer vision, classification, transformers and more models.

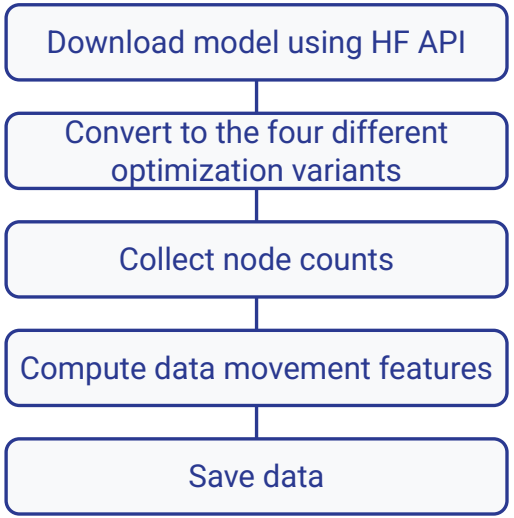
Hugging Face

Features

- graph features (133)
  - node counts (127)
  - data movement (6)
- hardware features (9)
- inference data (1)

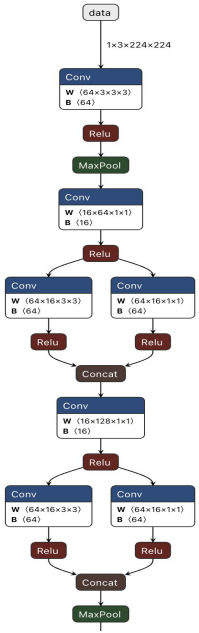
Collecting Graph Features

- 9,064 models after this stage



Node Counts

- How many times does a node appear in the model graph?
- 127 different node types considered



- conv_ops (3)
  - Eg: Conv, QLinearConv, ConvTranspose
- matmul_ops (8)
  - Eg: MatMul, Gemm
- elementwise_ops (28)
  - Eg: Relu, Sigmoid, Tanh, Mul, Div
- reduction_ops (15)
  - Eg: MaxPool, AveragePool
- normalization_ops (6)
  - Eg: Softmax, BatchNormalization
- movement_ops (19)
  - Eg: Transpire, Reshape
- other (48)
  - control flow, constants, shape transformations, boolean logic

Data Movement Features

- Compute-bound Operations
  - *conv_flops*
  - *matmul_flops*

- Memory-bound Operations
  - elementwise_mb
  - reduction_mb
  - Movement_mb
  - Normalization_mb

bytes = (input size + output size) × dtype_size

2: Collect Inference Data

Environment

- x86 Linux machines on BlueHive and Google Cloud
- Hyperthreading disabled
- Only CPU is used. No GPUs or accelerators.

Collecting Inference Data

- 16 runs with different combinations of core counts & memory

Bluehive

Run 1: 1 core, 16GB - bluehive
Run 2: 4 cores, 16GB - bluehive
Run 3: 8 cores, 32GB - bluehive
Run 4: 8 cores, 16GB - bluehive
Run 8: 4 cores, 8GB - bluehive
Run 10: 2 cores, 32GB - bluehive
Run 15: 4 cores, 32GB - bluehive
Run 13: 2 cores, 16GB - bluehive

Google Cloud

Run 5: 2 cores, 16GB - gcloud
Run 6: 1 core, 8GB - gcloud
Run 7: 1 core, 16GB - gcloud
Run 9: 4 cores, 32GB - gcloud
Run 11: 2 cores, 32GB - gcloud
Run 12: 4 cores, 16GB - gcloud
Run 16: 2 cores, 8GB - gcloud
Run 17: 6 cores, 32GB - gcloud

Collecting Inference Data

- Each model was downloaded (or loaded from cache) using the HF API.
- ORT_INTRA_OP_NUM_THREADS env variable decides how many cores ONNX Runtime uses.
- Input data was supplied by a random tensor based on the size of the input shape.
- Inference was captured using the ONNX Runtime CPU Provider. Each capture had 5 warm up runs, followed by 10 timed inferences.

Hardware Features

- memory_mb
- l1d_cache_kb
- l1i_cache_kb
- l2_cache_kb
- base_clock_mhz
- num_cores
- memory_bandwith_gbs
- memory_mb
- cpu_provider *(amd, intel)*

Final Dataset

- Total Features (143) = Node Counts (127) + Data Movement (6) + Hardware (9) + Inference Data

- 143,218 data points

warmup_runs	sample_runs	average_ms	stddev_ms	min_ms	max_ms	sample_1_ms	sample_2_ms	sample_3_ms	sample_4_ms	sample_5_ms	sample_6_ms	sample_7_ms
5	10	26.733058	0.154963	26.424997	26.989592	26.809016	26.816772	26.696212	26.736186	26.660548	26.582790	26.842040
5	10	28.205023	0.249364	27.827361	28.778841	28.282095	28.778841	28.189169	28.176415	28.130476	27.827361	28.220700
5	10	27.378879	0.199718	26.882358	27.629083	27.376523	27.267133	27.439609	27.408770	27.534673	27.426050	27.629083
5	10	27.955235	1.410857	26.744419	31.779969	27.222986	28.041914	27.712457	27.485469	28.273565	27.546022	26.744419
5	10	26.585971	0.063857	26.478669	26.692625	26.599434	26.575481	26.692625	26.590408	26.478669	26.609407	26.562819
5	10	27.579075	0.303379	27.082889	28.172198	27.674751	27.736353	27.617724	27.635601	27.455495	27.476915	27.735881
5	10	27.301601	0.280346	26.657908	27.628675	26.657908	27.310921	27.224551	27.628675	27.048872	27.419980	27.341580
5	10	27.460480	0.141997	27.324395	27.826788	27.525430	27.448730	27.495430	27.324395	27.494156	27.826788	27.439811
5	10	22.171580	0.433078	21.666280	23.026869	22.334734	22.321161	22.267085	22.095852	21.706037	23.026869	22.588885
5	10	22.694919	0.229937	22.466354	23.146766	22.585607	22.613418	23.004104	22.658717	22.681945	22.540450	22.597553
5	10	22.702008	0.463804	22.256312	23.700494	22.749030	22.805739	22.431755	22.360953	22.441621	23.256553	22.298923
5	10	22.034002	0.257200	21.716922	22.429028	22.093034	21.813568	21.831940	22.217591	21.898111	22.356669	21.784222
5	10	23.196055	0.254540	22.995983	23.693623	23.088836	23.693623	23.048365	23.648083	22.995983	23.077929	23.116806
5	10	39.421438	0.187125	39.150239	39.798880	39.798880	39.457327	39.464516	39.438132	39.404788	39.400488	39.587887
5	10	40.006155	0.266112	39.801785	40.526114	39.969799	39.981912	39.801785	39.806810	39.910596	40.468878	39.877834
5	10	23.095916	0.070066	22.988405	23.210453	23.210453	23.038764	23.060049	23.016681	23.118092	23.129975	23.125050
5	10	30.517546	1.187905	29.457743	33.760656	30.206949	30.151427	29.981892	30.062002	30.402097	30.791582	29.457743
5	10	36.434143	0.200359	36.206328	36.893012	36.471075	36.354393	36.627248	36.430925	36.206328	36.345231	36.288976
5	10	35.580726	0.166125	35.359237	36.005097	35.612660	35.508674	35.603357	35.489246	35.551297	35.586600	35.359237
5	10	35.591125	0.431903	35.272953	36.769542	35.388168	35.549424	35.660326	35.272953	35.279648	36.769542	35.421122
5	10	29.935625	0.232581	29.762015	30.550437	30.550437	29.976311	30.025892	29.881169	29.919841	29.762015	29.846157
5	10	36.249152	0.120639	36.097036	36.478102	36.289289	36.153223	36.168412	36.227371	36.277412	36.417109	36.159927
5	10	35.505223	0.252424	34.971292	35.726862	34.971292	35.651946	35.726862	35.113239	35.606901	35.618009	35.648836
5	10	35.541440	0.220073	35.211578	35.968182	35.494939	35.643032	35.554071	35.211578	35.599612	35.363796	35.287503
5	10	26.304991	0.053765	26.202165	26.376994	26.302717	26.376994	26.319603	26.235413	26.272998	26.331053	26.348592
5	10	27.062537	0.207464	26.780642	27.525719	27.186421	26.780642	27.135071	26.837643	27.087657	27.525719	27.106100
5	10	27.374803	0.301061	27.118099	28.164472	27.364912	27.118099	27.164457	27.296403	27.442812	27.392638	27.137478
5	10	26.874912	0.087441	26.733330	27.029454	26.959399	27.029454	26.843186	26.808454	26.855635	26.855826	26.924410
5	10	23.791594	0.269924	23.642566	24.548707	23.729019	23.642566	23.780938	23.685377	23.663083	23.702312	23.781037

Inference Data

mean:	201.87 ms
median:	63.36 ms
std dev:	**592.07 ms**
min:	0.03 ms
max:	43,674.17 ms
p10:	11.01 ms
p25:	25.81 ms
p75:	156.53 ms
p90:	399.78 ms
p95:	757.79 ms
p99:	2,555.19 ms

3: Train Models

Data Pre-Processing (1)

- Coefficient of Variation (std_dev / average_latency)
  - Data with CV >= 10% was removed from the final dataset
  - 6,382 rows (4.5% of data)

- Split into train/test/val on 70/15/15 split

- Numerical features (normalization_mb, etc.)
  - Rounded down, unnecessary precision removed

Data Pre-Processing (2)

- All numerical features were log scaled
  - Ensures sparse or large features do not influence training
- cpu_provider encoded as 0/1 for intel/amd (only categorical variable)
- Mean/Std. Dev normalized with StandardScaler

Data Pre-Processing (3)

- log_latency as the target
  - Latency prediction requires multiplicative error

- Model A
  - True latency (1 ms), Predicted latency (2 ms)
- Model B
  - True latency (1000 ms), Predicted latency (1001 ms)

- Both are off by 1ms, but Model A prediction is much worse!

Model Training

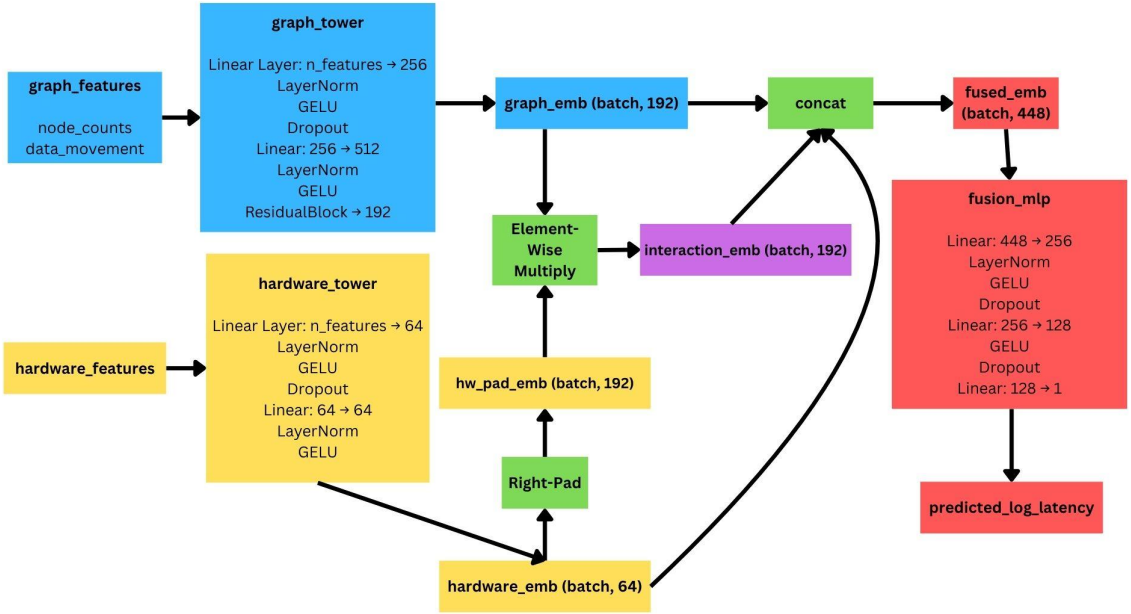
- Three different model architectures were tried
  - XGBoost, RandomForestRegressor, Neural Network

- Different Combinations of features
  - Removing node counts dropped performance significantly
  - Using a subset of features (Top-10 nodes) did not improve performance
  - Additional engineered features did not improve performance

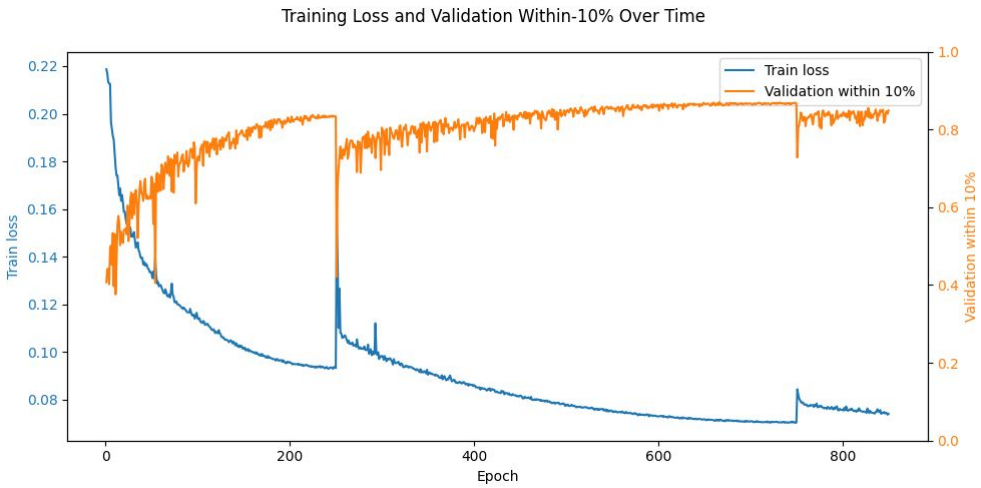
Architecture

“Two-Tower” Model

graph_emb, hardware_emb, interaction_emb go into fusion mlp



Training Process



850 epochs

Best checkpoint at Epoch 666

AdamW Optimizer w/ decay from 8e-4 to 2e-4

SmoothL1Loss

CosineAnnealingWarmRestarts every 250 epochs

Batch Size 1024

Total training time was ~20 minutes on Google Colab G4 GPU

4: Evaluation

Performance on Test Set

Model Type	Median Rel Err	p90 Rel Err	Median % Err	p90 % Err	Median Ratio	Within 10%	Within 25%
XGBoost	0.082	0.420	8.23%	42.06%	1.08	55.8%	80.8%
RandomForest	0.020	0.207	2.06%	20.73%	1.02	79.6%	92.1%
Neural Network	**0.023**	**0.132**	**2.00%**	**13.30%**	**1.02**	**87.3%**	**94.2%**
Random Guess (Mean)	-	-	218.59%	-	-	3.91%	9.46%
Random Guess (Median)	**-**	**-**	**74.25%**	**-**	**-**	**6.57%**	**15.12%**

Performance by Core Count

num_cores	count	within_10pct	within_25pct	within_50pct
1	3968.0	0.918347	0.963710	0.986895
2	6449.0	0.855947	0.946503	0.980462
4	6414.0	0.849860	0.918771	0.955098
6	1350.0	0.909630	0.930370	0.951852
8	2347.0	0.908394	0.933106	0.948871

Performance by CPU Provider

cpu_provider	total_count	miss_count	miss_rate
amd	8877.0	1534.0	17.2%
intel	11651.0	1072.0	9.2%

- A miss is a prediction outside 10% accuracy

cpu_provider	count	rmse_log	within_10pct	within_25pct	within_50pct
amd	8877.0	0.145405	82.7%	94.9%	98.1%
intel	11651.0	0.219293	90.7%	93.6%	95.9%

Performance by Platform

platform	cpu_provider	count	Miss Rate	within_10pct	within_25pct	within_50pct
bluehive	intel	10301.0	9.02%	90.9%	93.7%	96.0%
gcloud	amd	8877.0	17.2%	82.7%	94.9%	98.1%
gcloud	intel	1350.0	10.5%	89.4%	93.0%	95.2%

CLI Tool

- Provides prediction given an ONNX file with a confidence rating based on previous data.

```
(.venv)
    (.venv) /Users/sid-home/Projects/onnx-predict/tool git:(main*)
    ❯ ./predict_latency.py mnist.onnx \
  --cpu-provider intel \
  --l1d-cache-kb 32 \
  --l1i-cache-kb 32 \
  --l2-cache-kb 512 \
  --base-clock-mhz 2450 \
  --num-cores 2 \
  --memory-bandwith-gbs 205
variant                                                    average_ms  confidence
---------------------------------------------------------------------------------
mnist_disable_all.onnx                                       2.153985        high
mnist_basic.onnx                                             7.954413        high
mnist_extended.onnx                                         43.516480        high
mnist_enable_all.onnx                                       43.516480        high
(.venv)
```

```
    ❯ ./predict_latency.py resnet.onnx \
  --cpu-provider intel \
  --l1d-cache-kb 32 \
  --l1i-cache-kb 32 \
  --l2-cache-kb 512 \
  --base-clock-mhz 2450 \
  --num-cores 8 \
  --memory-bandwith-gbs 205
variant                                                    average_ms  confidence
---------------------------------------------------------------------------------
resnet_disable_all.onnx                                     52.133865        high
resnet_basic.onnx                                           52.133865        high
resnet_extended.onnx                                        52.001149        high
resnet_enable_all.onnx                                      51.361631         low
(.venv)
```

Limitations

- Scope: x86, Linux, CPU only
- Hard to test on unseen models since ONNX Model Zoo is so vast
- Inference data was collected from synthetic inputs and from limited set of hardware
- No peer works (with respect to ONNX models) to benchmark against

Future Work

- Expand to GPU inference
- Analyze poor performance on AMD machines
- Experiment with more/different hardware features

Thank you